

Uçtan Uca Büyük Veri Proje Yönetimi ve Mimari Uygulama Planı

1. Proje Vizyonu ve Mimari Strateji

Bu proje, akademik bir veri işleme çalışmasının ötesine geçerek, endüstriyel standartlarda yüksek erişilebilirlik (high availability) ve veri tutarlılığı (data consistency) prensiplerine dayalı bir veri mühendisliği ve veri bilimi hattı (pipeline) inşa etmeyi hedefler. Mimari stratejimiz, ham verinin kaynaktan alınmasından itibaren "schema evolution" ve ACID uyumluluğu gibi modern veri gölü (Data Lakehouse) gereksinimlerini karşılayacak şekilde tasarlanmıştır.

Apache Kafka ve Spark Structured Streaming entegrasyonu, sistemin yatayda ölçeklenebilirliğini garanti ederken; Delta Lake kullanımı, büyük veri ortamlarında karşılaşılan sürümleme ve güvenilirlik sorunlarını bertaraf eder.

Temel Hedeflerin Tanımlanması
Projenin ana odağı, "uçtan uca" (end-to-end) bir ekosistem kurmaktır. Bu kavram, verinin bir Python Producer aracılığıyla üretiminden (Ingestion), Spark ile işlenmesine (Processing), Delta Lake katmanlarında yapılandırılmasına (Storage) ve MLflow ile modelleme aşamasına (Intelligence) kadar olan tüm teknik derinliği kapsar. Buradaki temel başarı kriteri, her bileşenin bir diğeriyle tam entegre ve hatasız bir veri akışı sağlamasıdır.

Mimari Katmanların İnşası Sistem, birbirine sıkı sıkıya bağlı altı ana katmandan oluşur:

1. **Konteynerizasyon (Docker):** Servis izolasyonu ve ortam bağımsızlığı sağlar.
2. **Veri Üretimi (Kafka):** Gerçek zamanlı veri trafiğini yönetir.
3. **Veri İşleme (Spark):** ETL ve streaming süreçlerini yürütür.
4. **Depolama (Delta Lake):** Medalyon mimarisi ile veriyi katmanlandırır.
5. **Makine Öğrenmesi (MLlib & MLflow):** Deney takibi ve model yönetimini sağlar.
6. **Görselleştirme (Dashboard):** Analitik sonuçları iş değerine dönüştürür. Altyapı gereksinimlerinin karşılanması, bu katmanlı yapının sağlıklı bir şekilde ayağa kaldırılmasına bağlıdır.

2. Konteynerizasyon ve Altyapı Standartları

Büyük veri ekosistemlerinde servislerin birbirinden izole edilmesi ve ortamın her geliştirici için tekrarlanabilir olması bir zorunluluktur. Docker altyapısı, sistemin "tekrarlanabilir ortam" hedefine ulaşmasını sağlayarak, geliştirme ve üretim ortamları arasındaki uyumsuzlukları ortadan kaldırır.

Docker Altyapı Gereksinimleri Her bir mikro servis (Zookeeper, Kafka Broker, Spark, Python Producer) için **ayrı bir Dockerfile** oluşturulması zorunludur. Tüm sistem docker-compose.yml üzerinden yönetilmelidir.

Servis	Teknik Rol ve Zorunluluklar	Port / Volume Mapping
Zookeeper	Kafka broker koordinasyonu ve konfigürasyon yönetimi.	2181:2181
Kafka Broker	Mesaj kuyruğu ve streaming trafik yönetimi.	9092:9092 / Kafka_data
Python Producer	CSV'den veri okuyup Kafka'ya JSON basan özel konteyner.	Custom Dockerfile üzerinden
Apache Spark	Master ve Worker düğümleriyle veri işleme merkezi.	8080:8080 (UI), 7077 (Master)

Servis Entegrasyon Analizi ve Başarı Kriterleri Mimari açıdan, servislerin konteynerize edilmesi sistemin esnekliğini artırır. Başarı, sadece servislerin "Up" olmasıyla değil; Kafka broker'ın Zookeeper'a başarıyla bağlandığını gösteren loglar ve Spark executor'larının Master düğümüne sorunsuz register olmasıyla ölçülür. Dökümantasyonda bu bağlantı loglarının ekran görüntüleri yer almalıdır. Altyapı standartları sağlandıktan sonra, verinin sisteme gerçek zamanlı enjeksiyonu sürecine geçiş yapılır.

3. Gerçek Zamanlı Veri Akışı ve Streaming Stratejisi

Veri üretiminin durağan simülasyonlardan ziyade, gerçek dünya akış hızlarına (velocity) uygun tasarlanması stratejik bir önceliktir. Bu aşamada veri, rastgele üretilmek yerine belirlenmiş bir kaynaktan sistemli bir şekilde akıtılır. **Kafka Producer Parametreleri** Python tabanlı geliştirilecek olan Producer, seçilen veri setine ait **CSV dosyasını** satır satır okumalıdır. Mesajlar Kafka topic'ine aşağıdaki teknik standartlarda gönderilmelidir:

- **Format:** JSON (UTF-8).
- **Zorunlu Alanlar:** Timestamp (gönderim anı), Kullanıcı ID, Olay Tipi, İlgili ID.
- **Ayarlanabilir Hız:** Saniyede 10 ile 100 mesaj arasında dinamik olarak değiştirilebilir yapı.
- **İzlenebilirlik:** Producer logları üzerinden anlık gönderilen mesaj adedi takip edilmelidir. **Streaming İşleme ve Veri Kalitesi** Spark Structured Streaming, Kafka'dan gelen JSON akışını karşılayarak önceden tanımlanmış bir şema (schema definition) ile parse eder. Bu aşamada "Veri Güvenilirliği" (Data Reliability) sağlamak için; null değerlerin temizlenmesi, duplike kayıtların dropDuplicates() ile elenmesi ve format hatalarının (örn. tarih formatları) düzeltilmesi kritik önemdedir. İşlenen ve temizlenen bu veri akışı, kalıcı ve yönetilebilir bir yapı olan Delta Lake tablolarına aktarılır.

4. Delta Lake ile Veri Depolama ve Medalyon Mimarisi

Delta Lake, büyük veri göllerinde ACID uyumluluğu ve "Time Travel" (versiyonlama) yetenekleri sunarak veri yönetimini disipline eder. Projede uygulanacak Medalyon Mimarisi, Spark Structured Streaming ile doğrudan entegre çalışacaktır. **Katmanlı Yapı Analizi (Medallion Architecture)**

1. **Bronze (Tunç):** Kafka'dan gelen ham verinin şema tanımlanarak doğrudan yazıldığı katman.
2. **Silver (Gümüş):** Temizlenmiş, null analizi yapılmış ve standardize edilmiş veri katmanı.
3. **Gold (Altın):** İş mantığına göre agregasyonlar (summaries) içeren, analize hazır nihai tablolar. **EDA ve Özellik Mühendisliği (Feature Engineering)** Delta tabloları üzerinde yapılacak Keşifsel Veri Analizi (EDA) süreci; istatistiksel dağılımları ve trendleri ortaya çıkarmalıdır. Model başarısı için **en az 5 yeni özellik (feature)** üretilmelidir. Bu aşamada teknik dökümanda sadece kod değil, bu özelliklerin seçilmesinin arkasındaki **iş mantığı (business logic)** detaylıca açıklanmalıdır. Depolanan ve zenginleştirilen veri setleri, tahminleme süreçleri için makine öğrenmesi katmanına aktarılmaya hazırdır.

5. Makine Öğrenmesi Operasyonları ve MLflow Deney Takibi

Model geliştirme süreci, tüm hiper-parametrelerin ve metriklerin kayıt altına alındığı bilimsel bir deney süreci olarak yönetilmelidir. MLflow, bu süreçte şeffaflık ve model yaşam döngüsü yönetimini sağlar. **Model Seçim ve Karşılaştırma Matrisi** Problemin türüne göre aşağıdaki 5 modelin tamamının eğitilmesi ve karşılaştırılması zorunludur: | Problem Türü | Model Listesi (Zorunlu 5 Model) | Değerlendirme Metrikleri || ----- | ----- | ----- || **Sınıflandırma** | 1. Logistic Regression, 2. Decision Tree, 3. Random Forest, 4. GBT, 5. Naive Bayes veya Linear SVC | AUC-ROC, Accuracy, F1-Score, **Precision ve Recall**, Confusion Matrix || **Regresyon** | 1. Linear Regression, 2. Decision Tree, 3. Random Forest, 4. GBT, 5. Generalized Linear Regression (GLR) | RMSE, MAE, R², **Residual (Artık) Analizi** |

MLflow Entegrasyon Planı Tüm deneyler (parametreler, metrikler ve modeller) MLflow'a loglanmalıdır. Ayrıca, her model için "Feature Importance" analizi yapılarak, hangi değişkenin tahmine ne kadar katkı sağladığı raporlanmalıdır. Modellerin ürettiği analitik çıktılar, son aşamada karar vericiler için görsel bir arayüze dönüştürülür.